



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 06

NOMBRE COMPLETO: Robles Chávez Uri Shuhua Alexei

N° de Cuenta: 320195318

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 04 de Octubre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Descripción

En esta práctica se trabajó con diferentes métodos para aplicar texturas a objetos tridimensionales utilizando OpenGL y modelos creados en un programa de modelado 3D. El objetivo principal fue comprender cómo se relacionan las coordenadas UV de un modelo con una imagen de textura, así como comparar el proceso de construir un objeto directamente mediante código con el proceso de importar un modelo previamente texturizado.

Para el desarrollo se utilizaron las clases MeshModel, Texture y Model, además de shaders para representar los objetos dentro de la escena. En el proyecto se trabajó principalmente con posiciones XYZ, coordenadas UV y normales, ya que estos datos permiten definir la geometría, seleccionar qué parte de una imagen corresponde a cada superficie y determinar la orientación de cada cara. practica6

Los ejercicios realizados fueron:

- Creación y texturizado de un Holocrón mediante código.
- Importación de un Holocrón texturizado desde un programa de modelado.
- Importación de un avión con la cara de un personaje aplicada mediante texturas y logotipos en las alas.

Desarrollo

Ejercicio 1. Crear o modificar el Holocrón y texturizarlo por medio de código

Para el primer ejercicio se creó una nueva versión del Holocrón completamente mediante código. Para ello se agregó la función: void CrearHolocronStarWarsCodigo(). Esta función contiene directamente la información necesaria para construir el modelo: posición de los vértices, coordenadas UV y normales.

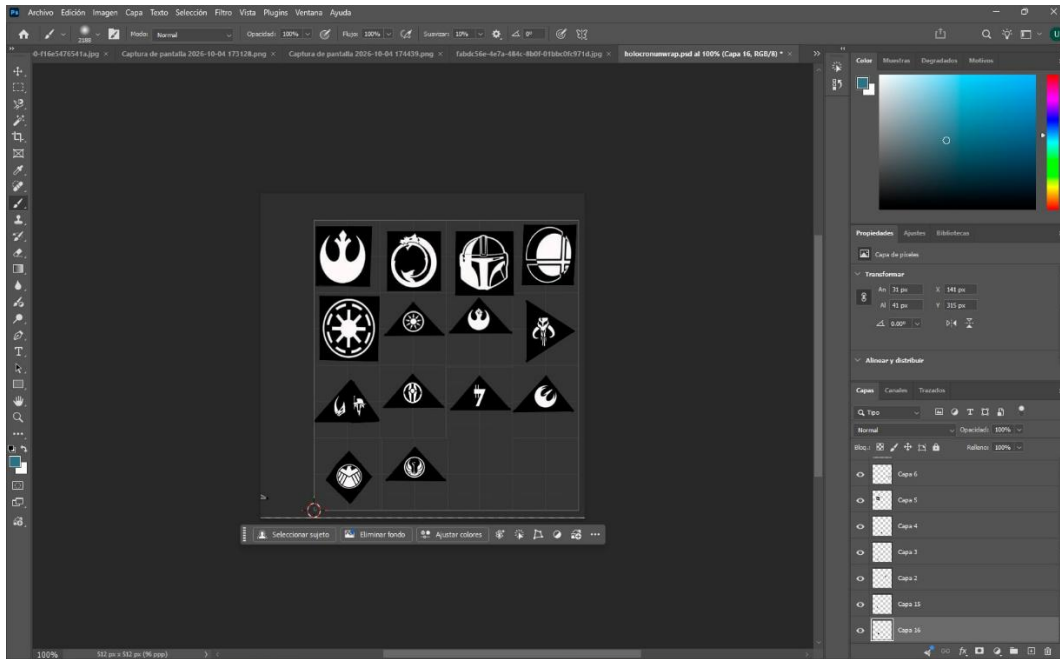


Figura 1. Textura creada con diferentes logotipos de Star Wars

La textura se carga dentro del programa mediante:

```
holocronStarWarsCodigoTexture = Texture("Textures/HolocronStarWars.jpg");
```

```
holocronStarWarsCodigoTexture.LoadTextureA();
```

Coordenadas UV del Holocrón definidas mediante código

A diferencia del Holocrón original de la práctica, en el cual las coordenadas UV todavía estaban establecidas en 0.0f, 0.0f, la nueva función contiene coordenadas UV específicas para cada vértice.

Cada vértice utiliza la estructura: X, Y, Z, U, V, NX, NY, NZ

De esta forma, cada punto de la geometría puede relacionarse con una posición concreta dentro de la textura.

Además, algunos vértices geoméricamente iguales se encuentran repetidos porque necesitan utilizar coordenadas UV o normales diferentes. Esto permite que una misma esquina del modelo pueda pertenecer a diferentes caras sin obligarlas a compartir la misma región de la textura.

```

267 //Ejercicio 1: reemplazar con sus datos de 6 caras texturizados, agregar normales
268 // average normals
269 GLfloat holocron_vertices[] = {
270     // front
271     //x          y          z          S          T          NX          NY          NZ
272     -2.517274,   -2.579095,   0.002115,   0.0f, 0.0f,   0.0f, 0.0f, 0.0f, //0
273     -1.185055,   -2.530499,   -1.198261, 0.0f, 0.0f,   0.0f, 0.0f, 0.0f, //1
274     0.049649,    2.440128,    -2.417217, 0.0f, 0.0f,   0.0f, 0.0f, 0.0f, //2
275     0.051152,    2.399781,    2.545043, 0.0f, 0.0f,   0.0f, 0.0f, 0.0f, //3
276     -2.646786,   -0.139047,    2.493757, 0.0f, 0.0f,   0.0f, 0.0f, 0.0f, //4
277     -1.328995,   -2.578386,    1.297838, 0.0f, 0.0f,   0.0f, 0.0f, 0.0f, //5
278     2.593003,    -0.085222,    2.505755, 0.0f, 0.0f,   0.0f, 0.0f, 0.0f, //6
279     1.385028,    -2.549476,    1.308677, 0.0f, 0.0f,   0.0f, 0.0f, 0.0f, //7
280     -0.140696,   -2.601934,    2.558561, 0.0f, 0.0f,   0.0f, 0.0f, 0.0f, //8
281     2.534944,    -0.061146,    -2.580280, 0.0f, 0.0f,   0.0f, 0.0f, 0.0f, //9
282     1.344182,    -2.542571,    -1.294387, 0.0f, 0.0f,   0.0f, 0.0f, 0.0f, //10
283     0.081432,    -2.513204,    -2.426005, 0.0f, 0.0f,   0.0f, 0.0f, 0.0f, //11
284     -2.453980,    -0.063388,    -2.477451, 0.0f, 0.0f,   0.0f, 0.0f, 0.0f, //12
285     2.632201,    -2.553892,    0.007226, 0.0f, 0.0f,   0.0f, 0.0f, 0.0f, //13
286     2.521206,    2.448241,     0.063388, 0.0f, 0.0f,   0.0f, 0.0f, 0.0f, //14
287     -2.451372,    2.421759,     0.077538, 0.0f, 0.0f,   0.0f, 0.0f, 0.0f, //15
288 };
289
290 MeshModel* holocron = new MeshModel();
291 holocron->CreateMeshModel(holocron_vertices, holocron_indices, 128, 84);
292 meshListModel.push_back(holocron);
293
294 }

```

Figura 2. Definición de vértices, coordenadas UV y normales utilizadas para construir y texturizar el Holocrón mediante código.

Índices utilizados para construir el Holocrón

También se definió el arreglo: unsigned int holocron_indices[]. Los índices determinan cómo se conectan los vértices para formar las caras triangulares del modelo. En esta nueva versión se utilizan 84 índices, equivalentes a 28 triángulos.

Posteriormente la geometría es enviada a CreateMeshModel()

```

233 void CrearHolocron()
234 {
235     unsigned int holocron_indices[] = {
236
237         12, 1, 0,
238         12, 4, 15,
239         14, 9, 2,
240         2, 12, 15,
241         8, 3, 4,
242         15, 4, 3,
243         3, 6, 14,
244         4, 5, 8,
245         6, 7, 13,
246         9, 10, 11,
247         11, 2, 9,
248         10, 7, 5,
249         13, 14, 6,
250         2, 3, 14,
251         2, 15, 3,
252         11, 12, 2,
253         13, 9, 14,
254         8, 6, 3,
255         12, 0, 4,
256         12, 11, 1,
257         4, 0, 5,
258         6, 8, 7,
259         9, 13, 10,
260         5, 0, 1,
261         1, 11, 10,
262         10, 13, 7,
263         7, 8, 5,
264         5, 1, 10
265     };
266 }

```

Figura 3. Índices utilizados para formar los triángulos del Holocrón y creación de la malla mediante CreateMeshModel.

Resultado del Holocrón texturizado mediante código

La función CrearHolocronStarWarsCodigo() se ejecuta durante la inicialización del programa junto con las demás geometrías. practica6

Posteriormente se coloca el objeto dentro de la escena y se aplica su textura



Figura 4. Resultado del Holocrón Star Wars generado mediante código y texturizado mediante coordenadas UV.

Ejercicio 2. Importar el Holocrón texturizado desde el programa de modelado

Para el segundo ejercicio se utilizó un flujo de trabajo diferente. En lugar de definir manualmente cada vértice y coordenada UV dentro de C++, se trabajó el modelo dentro de Blender.

En el programa de modelado se realizó el ajuste del UV Mapping utilizando la misma textura con logotipos de Star Wars y posteriormente el objeto fue exportado en formato OBJ.

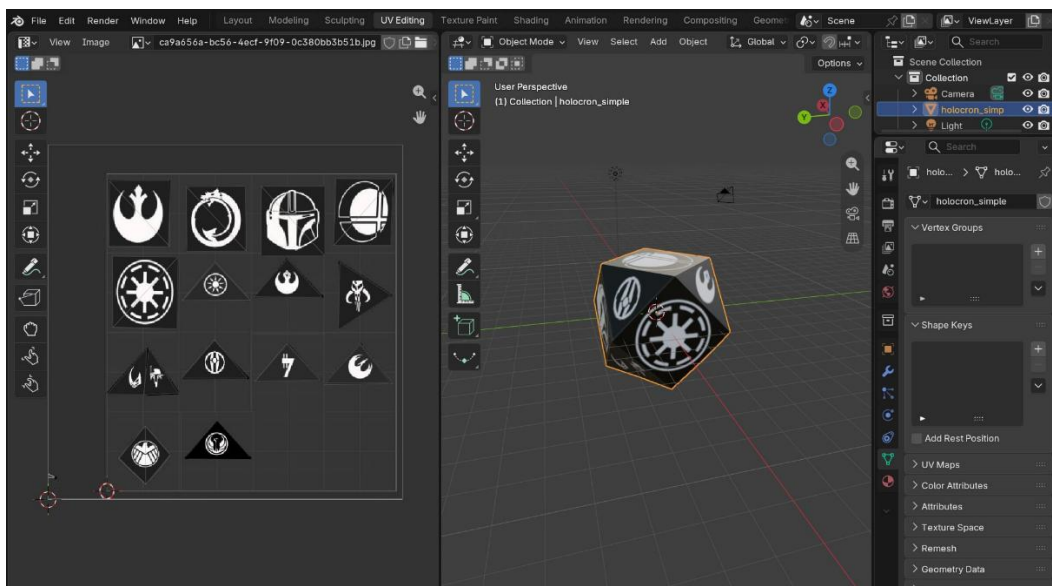


Figura 5. Vista previa del Holocrón texturizado en Blender antes de ser exportado al proyecto de OpenGL.

Importación del Holocrón en el código

El modelo exportado fue cargado dentro del proyecto mediante:
`HolocronStarWars_M.LoadModel("Models/HolocronStarWars.obj");`

Esto permite que la clase Model cargue la geometría, coordenadas UV y materiales asociados al modelo.

Después se ubica dentro de la escena:

```
model = glm::translate(model, glm::vec3(0.0f, 4.5f, -4.0f));
```

```
model = glm::scale(model, glm::vec3(0.35f, 0.35f, 0.35f));
```

```
HolocronStarWars_M.RenderModel();
```

```
429 // Ejercicio 2: cargar el cubo exportado con sus UV y material.  
430 Dado_M.LoadModel("Models/dadomodelo.obj");  
431  
432 Holocron_M = Model();  
433 Holocron_M.LoadModel("Models/holocron_simple.obj");  
434 HolocronStarWars_M.LoadModel("Models/HolocronStarWars.obj");  
435
```

Figura 6. Carga y renderizado del modelo OBJ del Holocrón Star Wars dentro del proyecto.



Figura 7. Comparación entre el Holocrón texturizado e importado desde Blender y la versión reconstruida mediante código.

Ejercicio 3. Importar un avión con la cara del personaje

Para el tercer ejercicio se trabajó con un modelo externo de avión.

El objetivo indicado en la práctica fue distribuir la textura de manera que:

- el vidrio frontal represente los ojos,
- la parte frontal del avión represente nariz y sonrisa,
- y las alas contengan logotipos relacionados con el universo del personaje.

Este requerimiento aparece directamente señalado en los comentarios del programa.



Figura 8. Imágenes utilizadas para texturizar las diferentes partes del avión.

UV Mapping del avión

Dentro de Blender se acomodaron las coordenadas UV de las diferentes partes del modelo sobre las imágenes correspondientes.

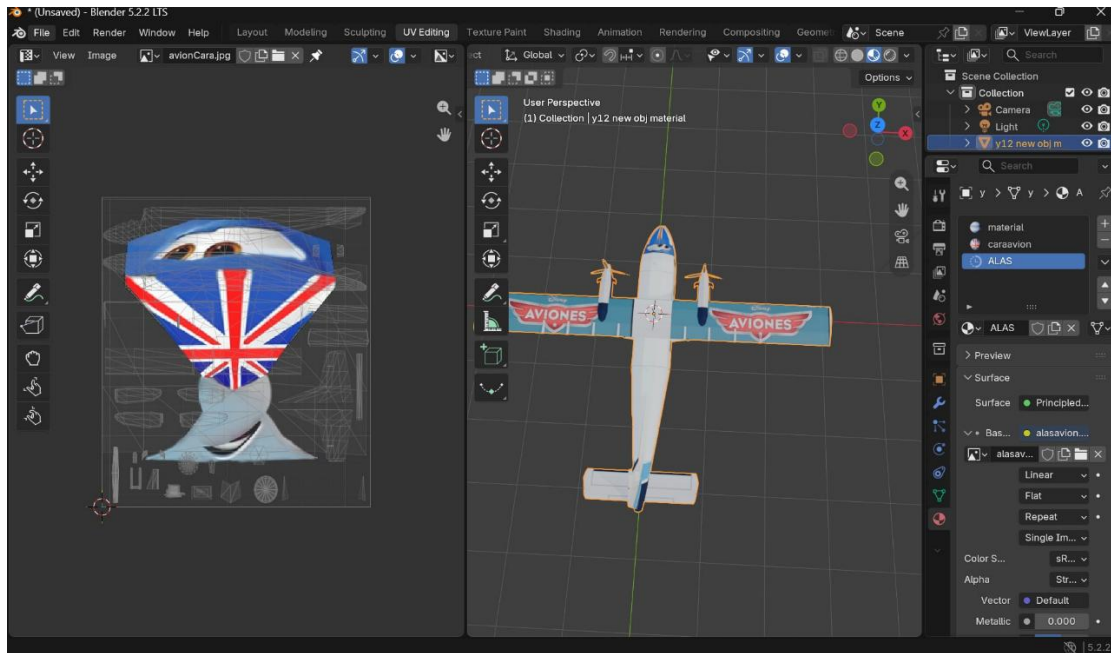


Figura 9. Resultado final del avión texturizado dentro del programa de modelado.

El modelo fue importado mediante: `AvionConCara_M.LoadModel("Models/avion con cara.obj");`

Antes de cargarlo se modifica temporalmente `GL_UNPACK_ALIGNMENT`:

`glPixelStorei(GL_UNPACK_ALIGNMENT, 1);` y posteriormente se restaura su valor anterior.

```

436 // Importar las tres texturas del avion
437 GLint alineacionAntesDelAvion = 4;
438 glGetIntegerv(GL_UNPACK_ALIGNMENT, &alineacionAntesDelAvion);
439 glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
440 AvionConCara_M.LoadModel("Models/avion con cara.obj");
441 glPixelStorei(GL_UNPACK_ALIGNMENT, alineacionAntesDelAvion);

```

Figura 10. Configuración de la alineación de píxeles y carga del modelo de avión texturizado.

Finalmente el avión se posicionó dentro de la escena mediante:

`model = glm::translate(model, glm::vec3(-4.5f, 2.0f, -5.0f));`

`model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));`

`AvionConCara_M.RenderModel();`



Figura 11. Modelo del avión texturizado e importado correctamente dentro de la escena OpenGL.

Resultado

Como resultado de la práctica se logró integrar dentro de una misma escena diferentes métodos de texturizado.

En el primer ejercicio se construyó un Holocrón mediante código utilizando vértices, índices, coordenadas UV y normales. En el segundo se trabajó el mismo concepto mediante un programa de modelado, exportando posteriormente el resultado como un archivo OBJ. Finalmente, en el tercer ejercicio se aplicaron varias zonas de textura sobre un avión para formar visualmente la cara de un personaje y añadir logotipos sobre las alas.

La comparación de los dos Holocrones permitió observar que el resultado puede obtenerse tanto mediante programación directa como mediante un programa de modelado, aunque el proceso de trabajo es diferente.



Figura 12. Resultado final de la Práctica 6 con los modelos texturizados integrados en la escena.

Problemas presentados

Durante la realización de la práctica se me presentaron principalmente dificultades relacionadas con el mapeo UV y la adaptación de las texturas a la geometría.

En el Holocrón fue necesario acomodar cuidadosamente las islas UV para que los diferentes logotipos quedaran centrados dentro de cada superficie. También observe que eliminar aristas de la geometría para facilitar el mapeo podía modificar la forma del objeto y producir deformaciones en la textura, por lo que fue necesario que conservara la triangulación del modelo y realizar los ajustes desde las coordenadas UV.

Otro problema que tuve fue que un mismo punto tridimensional podía necesitar distintas coordenadas UV dependiendo de la cara a la que perteneciera. Para solucionarlo, en la versión del Holocrón creada mediante código se utilizaron vértices repetidos con diferentes coordenadas UV y normales.

En el avión también fue necesario corregir la alineación de lectura de píxeles debido al tamaño de una de las imágenes RGB, utilizando `GL_UNPACK_ALIGNMENT` con un valor de 1 durante la carga del modelo.

Conclusión:

- a) La complejidad de esta práctica fue mayor que en ejercicios anteriores debido a que ya no se trabajó solamente con geometría, sino también con la relación

entre modelos tridimensionales e imágenes bidimensionales. El ejercicio del Holocrón generado mediante código fue el más complejo porque requirió representar manualmente para cada vertice. Además, fue necesario duplicar algunos vértices para permitir que una misma posición geométrica tuviera diferentes coordenadas UV.

El Holocrón importado simplificó este proceso, ya que Blender permitió modificar visualmente las islas UV y comprobar el resultado antes de exportarlo. Por último, el avión presentó un reto diferente porque fue necesario distribuir distintas partes de una imagen sobre zonas específicas del modelo para conseguir que el frente formara una cara reconocible.

- b) La práctica me permitió comprender que el texturizado no consiste solamente en aplicar una imagen sobre un objeto, sino en establecer una relación precisa entre las coordenadas del modelo y las coordenadas de la textura. También me permitió comparar dos formas de trabajo utilizadas en gráficos por computadora: la construcción directa mediante código y el uso de herramientas externas de modelado.
- c) Conclusión: Al finalizar la práctica se logró aplicar correctamente el concepto de mapeo UV a diferentes modelos tridimensionales. La creación del Holocrón mediante código permitió comprender cómo se relacionan los vértices, índices, coordenadas UV y normales dentro de una malla. La importación del mismo modelo desde Blender permitió comprobar cómo esta información puede generarse de forma visual y posteriormente ser utilizada dentro de OpenGL, asimismo, el ejercicio del avión permitió aplicar el texturizado de una manera más creativa, utilizando diferentes zonas del modelo para representar los ojos, nariz, sonrisa y logotipos.

Bibliografía

Alejandro D. (2025). *Como poner TEXTURAS en BLENDER FÁCIL || UV y tips* [Video]. YouTube. <https://www.youtube.com/watch?v=tAlJ57S0hEg>